

OpenAI Codex Blue Book: From Beginner to Architect (The Codex Blue Book)

Author: Hunk Wu

Ch.01 Saying Goodbye to Handwritten Code: Product Mindset in the Era of Vibe Coding

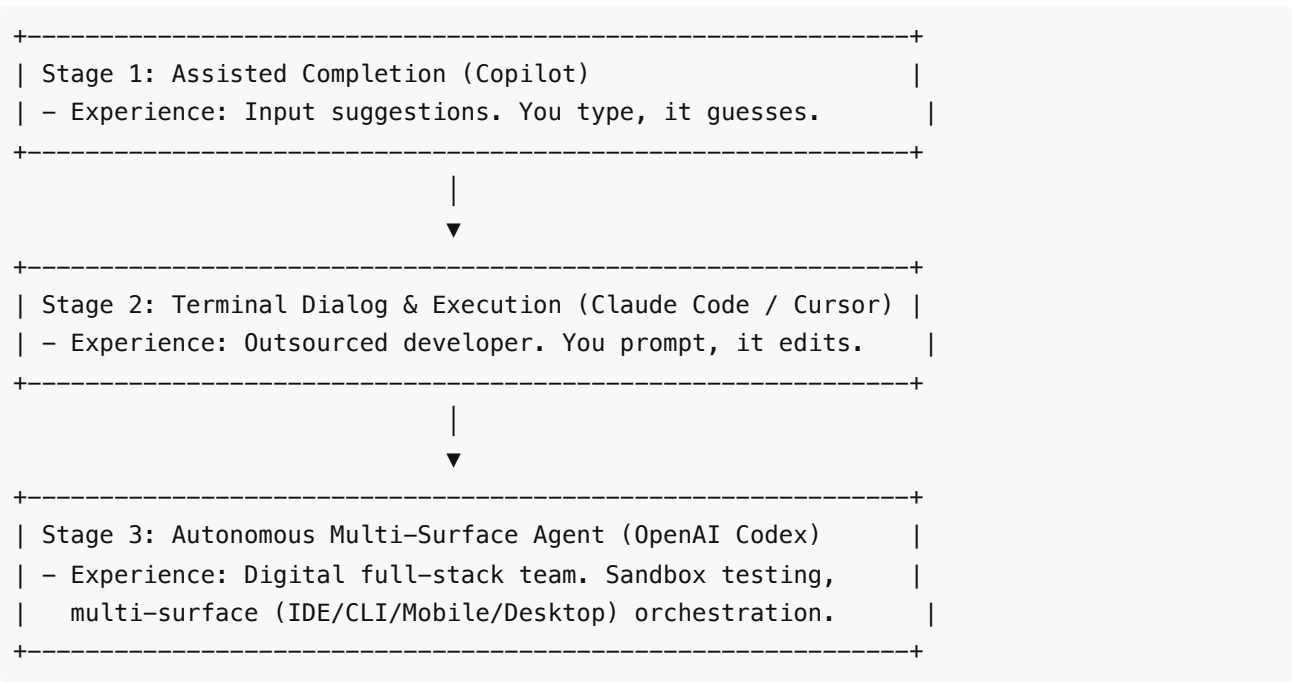
 *"Humans should no longer manually write boilerplate code. Your hands belong on the steering wheel, not the pushcart."*

When chatting with readers of "实战产品说" (Real-World Product Talk), I often notice a common pitfall: developers pushing themselves to memorize AI coding commands and shortcuts as if they were cramming for an API manual.

Wake up! In the era of the modern OpenAI Codex platform, the barrier to code generation has dropped to zero. This is called **"Vibe Coding"**—where you focus on product core logic, commercial viability, and user experience, while leaving the heavy lifting to autonomous agents.

This chapter will help you shift your mindset from a "code typist" to an "AI orchestrator."

1.1 The Evolution of AI Coding: Where Are We?



Codex has pushed us into Stage 3:

- **Multi-Device Orchestration:** Compile locally via CLI, monitor builds on the go via ChatGPT Mobile, and run visual browser audits via Desktop Computer Use.
- **Reasoning Core:** Backed by reasoning models (like o-series), Codex excels at long-horizon planning, resolving dependency blockages and config failures on its own.

1.2 The Core Shift: From Process Control to Boundary Control

As a Product Manager (PM), you know that when writing a PRD, you don't instruct developers on "how to structure their loops." You define acceptance criteria and boundary constraints.

With Codex, the same rule applies. Stop micromanaging how AI writes logic. As an orchestrator, focus on three things:

1. **Define the End State (Goal):** What problem are we solving?
2. **Establish Guardrails (Constraints):** Which modules are off-limits?
3. **Automate Validation (Validation):** Write test suites so Codex can prove the work is correct.

*[!IMPORTANT] **Orchestrator Formula** Successful Release = Clear Goal + Strict Constraints + Automated Testing*

1.3 Your New Moat: Orchestration and Domain Insight

When code is commoditized, what is your moat? As shared on pmer.cn: **Your value lies in defining business boundaries and empathizing with user pain points.** Your job is to structure architecture with Codex, deploy fast, and let the market validate your MVP.

Ch.02 Cross-Device Control: Building Your Codex Multi-Surface Productivity Matrix

In the AI-Native era, setting up your control room stably is rule number one. Many beginners rush into prompting without setting up local configs correctly, causing the agent to burn tokens on simple permission errors.

This chapter walks you through configuring the Codex CLI, Desktop App, and ChatGPT Mobile app integration.

2.1 Installing and Configuring the CLI (Rust Core)

The Codex CLI client is built in Rust for top performance.

1. Environment Verification

Make sure you have Rust tooling installed:

```
rustc --version
curl -fsSL https://codex-agent.download/install.sh | sh
```

2. Injecting API Keys and Setting Up Billing Guardrails

```
export OPENAI_API_KEY="sk-proj-xxxxxx..."
export CODEX_MAX_BUDGET_PER_TASK=2.0
export CODEX_MAX_RPM=50
```

 **Product Tip:** Always set a monthly hard limit (e.g., \$50) in your OpenAI developer dashboard as a failsafe.

2.2 Desktop App & Computer Use Permissions

To allow Codex to run visual audits, configure system accessibility permissions on macOS:

- **Accessibility:** Allows Codex to simulate mouse clicks and key inputs.
- **Screen Recording:** Allows the Vision model to analyze layouts.
- **Full Disk Access:** Enables file syncing between local workspaces and sandboxes.

2.3 Mobile Webhook Integration

Configure a webhook in `.codex/config.json` to push status alerts and deployment gates to your phone (via WeChat, Slack, or Telegram):

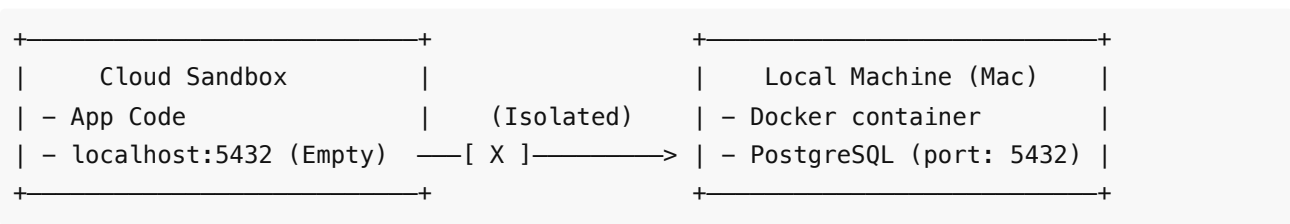
```
{
  "telemetry": {
    "enabled": true,
    "webhook_url": "https://api.pmer.cn/webhook/hunkwu-push",
    "notify_on": ["failed", "awaiting_auth"]
  }
}
```

Ch.03 Breaking the Cloud Island: Sandbox Debugging and Deep Local Environment Tunneling

One of the most common developer complaints is: *"Why does Codex keep saying 'Connection refused to localhost:5432' when I have PostgreSQL running in Docker?"*

This happens because Codex runs in an isolated cloud sandbox. `localhost` inside the sandbox refers to the container itself, not your local machine.

3.1 The Sandbox Barrier



To connect the cloud sandbox to your local database, we must build a reverse tunnel.

3.2 Setting Up Reverse Tunnels via SSH or Ngrok

Method 1: SSH Port Forwarding (Recommended)

If you have a public VPS, run this locally:

```
ssh -R 54320:localhost:5432 user@your-public-vps.com -N
```

Then configure your database URL in the sandbox env to target the VPS port:

```
DATABASE_URL="postgresql://postgres:password@your-public-vps.com:54320/dev_db"
```

Method 2: Ngrok (Fast Zero-Config Option)

Run locally:

```
ngrok tcp 5432
```

And bind the public TCP tunnel address provided by Ngrok to your sandbox configuration:

```
DATABASE_URL="postgresql://postgres:password@0.tcp.ngrok.io:12345/dev_db"
```

3.3 Directory and Environment Variable Isolation

Never sync `.env` files containing raw secrets to the sandbox. Keep them in `.gitignore` and add strict instructions to your [AGENTS.md](#):

```
## 🛑 Hard Constraints
- Never sync or commit files matching *.env.
- Access configurations through src/config.ts.
```

Ch.04 Goal-Driven Engineering: Taming Reasoning Agents with Boundaries and Assertions

In the era of reasoning models like o3 and GPT-5.5, step-by-step prompts constrain the AI. Reasoning models possess immense internal planning space—over-specifying instructions only limits their efficiency.

4.1 Concept: Don't Tell a Michelin Chef How to Chop

Avoid micro-managing the process: ❌ *"Import Express, import Axios, write a try-catch block, print errors via console.log."*

Instead, practice **Goal-Driven Engineering**: ✅ *"Create a GitHub OAuth route. Read GITHUB_ID from the env. Ensure Vitest tests pass."*

Define the **Goal**, establish **Constraints**, and provide **Validation Specs**—leaving execution details to the agent.

4.2 The Goal-Driven Specs Template

```
# 🎯 Goal
[Define the desired end state clearly]

# 🛑 Constraints
- [List safety, architectural, and dependency constraints]

# ✍️ Validation Specs
- [Specify automated test scripts and verification criteria]
```

4.3 Case Study: Rate-Limited Express Gateway

❌ Procedural Prompt (Too Restrictive)

"Write an Express proxy. Install express-rate-limit. Set windowMs to 15m. Proxy requests to github using axios and append bearer token."

✅ Goal-Driven Specs (Recommended)

🎯 Goal

Implement an Express API proxy routing requests to GitHub.

🛑 Constraints

- Must use Redis as the store for rate-limiting (no memory store).
- Hard limit proxy timeouts to 3000ms.
- Load secrets securely from process.env.GH_TOKEN.

✅ Validation Specs

- Return 429 Too Many Requests when limits exceed 60 req/min.
- Return 504 Gateway Timeout on proxy network errors.

Ch.05 Defining the CAP Protocol: Building Your Project's AGENTS.md Rule Compliance Layer

Without clear boundaries, autonomous agents can become overly creative, rewriting legacy modules and breaking coding styles.

To maintain code hygiene, we establish `AGENTS.md` in the project root as our **Codex Collaboration Protocol (CAP)**.

5.1 Why AGENTS.md?

1. **Context Restore:** Restores the tech stack, rules, and commands automatically on every startup.
2. **Anti-Corruption:** Segregates editable domains from read-only core scripts.
3. **Command Safety:** Constraints testing and deployment pipelines to avoid dangerous command execution.

5.2 AGENTS.md Structure

Project Fingerprint

- Explains active frameworks, directories, and tech stacks.

Developer Commands

- Defines valid scripts for building, testing, and db migrations.

Styles & Architecture Patterns

- Lists coding standards (e.g. Hooks in @/hooks, UI in @/components/ui).

Agent Boundary & Hard Rules

- Defines read-only paths and required checks before PR creation.

5.3 A Production-Grade AGENTS.md Template

Find our reference template directly on the home page: [AGENTS.md](#).

Ch.06 Correcting Course: Supervising the CoT Reasoning Chain Like a Tech Lead

Reasoning models plan their tasks through Chain of Thought (CoT). By monitoring CoT logs, you can spot logical errors and intervene before the agent runs into loops.

6.1 Inspecting CoT Logs in Real-time

Expose the internal draft logic using:

```
codex --verbose --show-cot
```

Log Telemetry Example:

```
[Thinking Process]
- Checking src/lib/redis.ts...
- Found `redis = new Redis()` without retryStrategy.
- Hangups violate 3s timeout constraint.
- Solution: Add custom retryStrategy.
```

6.2 Spotting AI Failure Loops

Watch for these red flags:

1. **Dependency Loop:** AI installs packages, fails due to peer conflict, tries to force-install, fails, and switches to another package in an infinite loop.
2. **Regression Loop:** AI edits A, breaking test B; edits B to fix it, breaking C; edits C, breaking A again.

6.3 Intervention: Stop, Refine, and Rollback

Step 1: Interrupt (`Ctrl + C` or `stop`)

Freezes the sandbox state.

Step 2: Refine (`refine`)

Redirect the agent's logic:

```
codex refine "Do not install axios-retry. Use native AbortController instead."
```

Step 3: Rollback

If the code is corrupted, revert using git:

```
git checkout -- src/lib/redis.ts
```

Add a constraint in `AGENTS.md` to prevent similar actions.

Ch.07 Closing the Visual Loop: Automated Auditing and Design Verification with Desktop Computer Use

With **Computer Use**, Codex can view your screen, open Chrome, inspect DevTools, and adjust CSS alignment directly.

7.1 Safety Bounds & Coordinate Targeting

Restrict screen control to virtual displays or specific apps:

```
{
  "computer_use": {
    "allowed_applications": ["Google Chrome"],
    "viewport_restriction": { "width": 1280, "height": 800 }
  }
}
```

The agent captures the viewport, runs layout analyses, and targets elements by pixel coordinates (x, y).

7.2 Visual QA: Figma Mockup Verification

Let's instruct Codex to align our local login page with a design export.

```
# 🎯 Goal
Align local /auth/login layout with figma_login_mockup.png.

# 🚀 Codex Steps
1. Launch Google Chrome on http://localhost:3000/auth/login.
2. Capture screenshot of the render area.
3. Compare layout dimensions with the Figma mockup.
4. Correct CSS discrepancies (e.g. increase top padding from 16px to 32px).
```

The agent modifies the Tailwind classes in `page.tsx`, reloads Chrome, and verifies layout correctness.

Ch.08 Mobile Sentinel Workflows: 24/7 Remote Development and Orchestration

Don't chain yourself to your terminal. Set up a remote watchtower to approve deployments from your phone.

8.1 Watchdog Pipeline Setup

Configure GitHub Actions to verify builds and notify your phone on failure:

```
# .github/workflows/codex-watchdog.yml
# ...
- name: Push Fail Notice to Mobile
  if: env.STATUS == 'failed'
  run: |
    curl -X POST -H "Content-Type: application/json" -d '{"message":
"Codex sandbox testing failed."}' "${{ secrets.MOBILE_WEBHOOK_URL }}"
```

8.2 Remote Command Authorization

When Codex prepares a production deploy, let it hold and query your gateway. Reply `1` (Approve) or `0` (Abort) via WeChat or Slack:

```
// Express Handler excerpt
app.post('/api/mobile-reply', (req, res) => {
  const { userMessage, user } = req.body;
  if (user !== 'hunkwu') return res.sendStatus(403);

  if (userMessage === '1') {
    exec('echo "approved" > /tmp/codex_deploy_signal');
    res.json({ reply: '🚀 Deploy approved!' });
  } else {
    exec('pkill -f codex && git checkout -- .');
    res.json({ reply: '🛑 Aborted and rolled back.' });
  }
});
```

Ch.09 Codebase Revitalization: Reverse Engineering and Progressive Decoupling of Legacy Systems

Don't rewrite legacy systems from scratch. Decouple them progressively using characterization tests.

9.1 Mapping the Database

Let Codex parse Prisma configurations to generate a visual database topology:

```
# 🎯 Goal
Analyze the database configuration and generate a Mermaid ERD.
```

9.2 Writing Characterization Tests

Lock down current API endpoint behavior before refactoring:

🎯 Goal

Write baseline integration tests for `src/pages/api/checkout.ts`.
Ensure zero modifications to the route logic during test setup.

9.3 Decomposing Logic

With tests verifying behavior, tell Codex to extract fat controller logic into modular services:

🎯 Goal

Extract pricing calculations from `checkout.ts` to `services/discountService.ts`.

✍️ Validation Specs

— Run tests and ensure 100% pass rates.

Ch.10 Monetization in Practice: Shipping a Commercial SaaS MVP in 2 Hours

10.1 Prisma Database Schema

Instruct Codex to generate models for users and Stripe subscriptions:

```
model User {
  id          String          @id @default(uuid())
  email       String          @unique
  subscription Subscription?
}

model Subscription {
  id          String          @id @default(uuid())
  userId      String          @unique
  stripeSubId String          @unique
  status      SubscriptionStatus
  currentPeriodEnd DateTime
  user        User            @relation(fields: [userId], references: [id],
onDelete: Cascade)
}
```

10.2 Handling Stripe Webhooks

Have Codex write a secure Webhook handler:

```
// src/app/api/webhooks/stripe/route.ts
export async function POST(req: Request) {
  const body = await req.text();
  const signature = req.headers.get('stripe-signature')!;
  let event = stripe.webhooks.constructEvent(body, signature,
process.env.STRIPE_WEBHOOK_SECRET!);

  if (event.type === 'checkout.session.completed') {
    // Upsert subscription status to 'ACTIVE' in database
  }
}
```

Ch.11 Mobile Extension: Expo Cross-Platform App Development and Cloud Packaging

Build iOS and Android apps without configuring Xcode or Android Studio locally.

11.1 Expo Router Setup

Ask Codex to initialize the React Native project structure:

```
# 🎯 Goal
Bootstrap a React Native project using Expo Router and NativeWind (Tailwind CSS).
```

11.2 Native Dependency Conflict Resolution

If Cocoapods auto-linking fails in the sandbox, instruct Codex to align packages using Expo SDK CLI:

```
codex refine "Reinstall the package using 'npx expo install' to match SDK versions."
```

11.3 EAS Cloud Builds

Trigger cloud compilation remotely without local environment headaches:

```
eas build --platform ios --profile production --non-interactive
```

Ch.12 The Final Frontier: Building an Automated Growth Flywheel for a One-Person SaaS

Building the product is only half the battle. You must build an automated acquisition funnel.

12.1 SEO Content Automation

Write a Node.js script to automatically generate your static sitemaps as new blog posts deploy:

```
// scripts/generate-sitemap.js
const fs = require('fs');
const urls = ['/', '/auth/login', '/features', '/blog/introducing-our-saas'];
const xml = `<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  ${urls.map(url => `<url><loc>https://pmer.cn${url}</loc></url>`).join('')}
</urlset>`;
fs.writeFileSync('public/sitemap.xml', xml);
```

12.2 Daily Stripe Briefings

Get daily performance updates sent directly to your phone:

```
// daily-report.js
const userCount = await prisma.user.count({ where: { createdAt: { gte: yesterday } } });
const activeSubs = await prisma.subscription.count({ where: { status: 'ACTIVE' } });
await axios.post(process.env.MOBILE_WEBHOOK_URL, {
  text: `昨日新增用户: ${userCount}人 | 活跃订阅: ${activeSubs}个`
});
```

12.3 The Ultimate Moat

Code is commoditized. Your real moats are **domain expertise** and **empathy for the user**. Keep shipping, keep optimizing, and stay close to your audience.
